MACHINE LEARNING PROJECT – 1

Team Name: Outliers

Regression and Classification Report

Hetul Mehta (MT2026036) | Vanshit Shah (MT2026112)
Girish Chougule (MT2026028) | Yashik Jain (MT2026117)

GitHub: https://github.com/mehtahetul/ML-Project-1

Contents

Part I: Domestic Flight Price Prediction Challenge **2**
1 Introduction . . . 2
2 Problem and Dataset . . . 2
3 Exploratory Data Analysis (EDA) . . . 3
4 Preprocessing and Feature Engineering . . . 4
5 Cross-Validation Framework . . . 5
6 Models Tried . . . 6
7 Hyperparameter Tuning . . . 9
8 Final Blend and Exact-Match Lookup . . . 9
9 Results and Discussion . . . 10
10 Best Model . . . 12
11 Key Insights and Limitations . . . 12
12 Conclusion . . . 12

Part II: Vehicle Type Classification **13**
1 Introduction . . . 13
2 Problem and Dataset . . . 13
3 Exploratory Data Analysis (EDA) . . . 13
4 Preprocessing . . . 16
5 Train–Validation Split . . . 17
6 Models Tried . . . 17
7 Results and Discussion . . . 20
8 Best Model . . . 22
9 Key Insights and Limitations . . . 23
10 Conclusion . . . 23

PART I

REPORT: Regression

Domestic Flight Price Prediction Challenge

Hetul Mehta (MT2026036) | Vanshit Shah (MT2026112)

1 Introduction

This part predicts the ticket price of a domestic flight, in Indian Rupees, from the booking details: airline, flight number, route, departure and arrival time, number of stops, class, duration and days left before departure. The task is the Domestic Flight Price Prediction Challenge on Kaggle.

The main difficulty is that the data mixes two very different markets. Business fares are on average about 7 times higher than Economy fares and far more spread out, so a small number of expensive Business tickets dominate any squared-error metric. We therefore compared several ways of modelling the price (log, raw price, Tweedie and Gamma) and of handling the two classes (one model for both, or one model per class).

The work covers exploratory data analysis (EDA), one shared feature pipeline, eleven models (four linear, seven tree-based) scored on the same 5-fold cross-validation (CV), a final average of the six best tree models combined with an exact-match lookup, and an analysis of where the remaining error comes from.

At a glance

Data:	15,589 training rows, 3,898 test rows, 10 input columns, target `price`
Models tried:	11 (four linear models, seven tree-based models)
Final model:	Average of six tree models × 1.01, plus exact-match lookup
Final CV score:	RMSE 9,877 MAE 3,167 R^2 0.9363

2 Problem and Dataset

The task is to predict `price` for every test row. The training set has 15,589 rows and 12 columns (including `id` and `price`); the test set and the sample submission each have 3,898 rows.

Target. `price` ranges from Rs. 3,801 to Rs. 323,177, with mean Rs. 29,817, median Rs. 10,183 and standard deviation Rs. 39,128. The mean is about three times the median, so the distribution is strongly right-skewed.

Inputs. Table 1 lists the ten input columns. Eight of them are categorical, so encoding them well is a central part of the pipeline. `id` is a record number and is not used.

Table 1: Input columns.

Group	Columns
Carrier	`airline` (18 airlines); `flight`, a code such as `6E 285` (1,070 codes in train), whose part before the space is the *flight prefix*
Route and schedule	`source_city`, `destination_city` (6 cities each); `departure_time`, `arrival_time` (6 time-of-day slots, Early_Morning to Late_Night)
Itinerary	`stops` (zero, one, two_or_more), `duration` (hours), `flight_class` (Economy or Business)
Booking time	`days_left`, days between booking and departure (1 to 57)

3 Exploratory Data Analysis (EDA)

3.1 Price and Flight Class

Raw prices are heavily right-skewed; on the log scale they split into an Economy group and a Business group (Figure 1). Business fares are about 7 times higher on average and about 10 times more spread out (Table 2). Business tickets therefore dominate any squared-error metric, which is why we tested separate models per class.

Table 2: Price by flight class (training data).

Class	Rows	Mean price (Rs.)	Std (Rs.)
Business	6,380	60,127	46,415
Economy	9,209	8,818	4,670

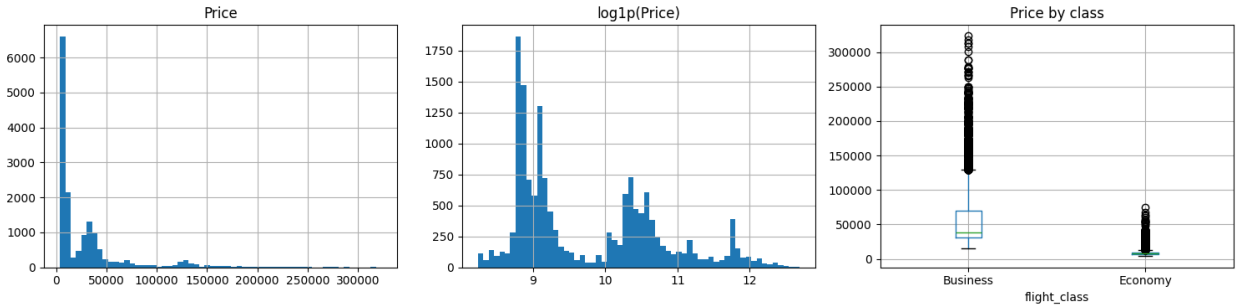


Figure 1: Price distribution: raw price (left), log(1 + price) (centre) and price by class (right).

3.2 Main Price Drivers

- **Stops:** mean price is Rs. 15,508 for non-stop, Rs. 62,782 for one stop and Rs. 130,463 for two or more stops. Part of this comes from long one-stop Business trips.
- **Booking time:** booking in the last 7 days costs 1.14 times the average of earlier bookings in Business and 1.08 times in Economy. Before that, the trend is noisy (Figure 2, left).
- **Departure time:** it shifts the mean price in both classes, by different amounts in each (Figure 2, centre).
- **Airline:** mean Business price varies widely, from Oceanic Airlines and Wayfarer Aviation at the low end to Weyland Air at the high end (Figure 2, right). Airline and class together carry much of the price signal.

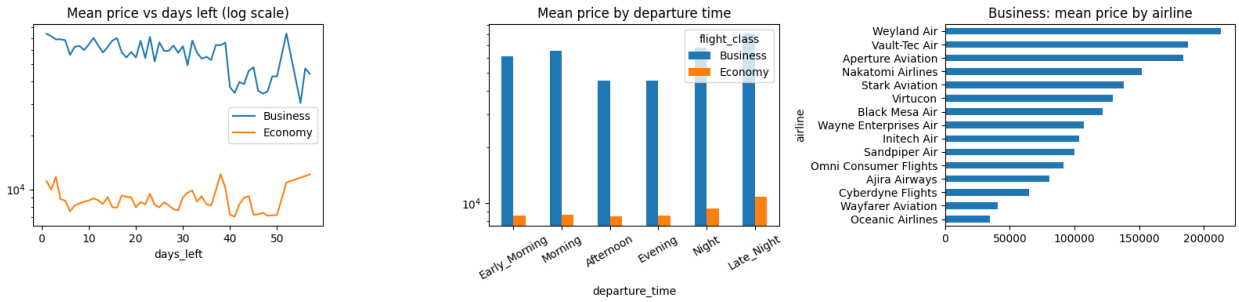


Figure 2: Mean price by days left (left, log scale), by departure time and class (centre, log scale), and mean Business price by airline (right).

3.3 Missing Values and Repeated Rows

Only two columns have missing values, at similar rates in train and test: `airline` (3,089 in train, 19.8%; 799 in test) and `duration` (3,095 in train, 19.9%; 776 in test). Both are easy to fill (Section 4):

- each of the 20 flight prefixes belongs to exactly one airline, so the prefix gives the missing airline;
- 97.8% of missing durations can be copied from other rows with the same flight, route and stops.

No other column has gaps, so no rows were dropped. 314 training rows repeat another row's inputs; they were kept, and Section 8.2 makes direct use of this repetition.

3.4 Rare Categories and Flight Codes

Two airlines dominate (Wayfarer Aviation 5,200 rows, Oceanic Airlines 5,112), while six have fewer than 10 rows, so anything learned about them is noisy. Only 66 rows have two or more stops. Only 0.9% of test rows have a flight code unseen in train, so flight-level information is almost always available. However, the same flight in the same class has no fixed price: among flights with at least 5 rows, the median ratio of standard deviation to mean price is 0.161 in Business and 0.150 in Economy, and reaches 0.650 and 0.943. This spread limits how accurate any model can be (Section 9).

4 Preprocessing and Feature Engineering

The shared steps below use only the input columns of train and test, never the price, so they cannot leak the target. Every step that is fitted to data (encoders, scalers, splines) is fitted inside each CV fold on training rows only.

4.1 Shared Steps (Used by Every Model)

- **Integrity check:** rows missing any core column (flight, cities, times, stops, class, days left, price) would be dropped. None were found, so all 15,589 rows were kept.
- **Missing airline:** filled from the flight prefix, which maps to exactly one airline. A flag `airline_missing` marks filled rows.
- **Missing duration:** filled with the median duration of the first matching group in this order: (1) same flight, route and stops; (2) same airline, route and stops; (3) same route and stops; (4) same stops; (5) all rows. A flag `duration_missing` marks filled rows. The medians use train and test inputs only.
- **Ordinal codes:** `stops_n` (0, 1, 2), `dep_n` and `arr_n` (Early_Morning = 0 to Late_Night = 5), `is_business`.
- **New features:** `flight_prefix` and numeric `flight_num` from the flight code; combinations `route`, `route_class` and `airline_class`; `dur_per_stop` = duration / (stops + 1); `days_bucket` (edges at 2, 7, 14, 21, 35 days); `last_minute` ($\leq$ 7 days left); `flight_count`, the number of rows with the same flight code.

This gives 24 features and no missing values (Table 3).

Table 3: Features used by the tree-based models.

Type	Count	Features
Categorical	11	airline, flight_prefix, source_city, destination_city, route, route_class, airline_class, departure_time, arrival_time, flight_class, stops
Numeric	13	flight_num, duration, dur_per_stop, days_left, days_bucket, last_minute, stops_n, dep_n, arr_n, is_business, airline_missing, duration_missing, flight_count

4.2 Model-Specific Steps

- **XGBoost and LightGBM:** both libraries handle the 11 categorical columns directly, so no one-hot encoding is needed.
- **Decision Tree and Random Forest:** scikit-learn needs numeric inputs, so categorical columns are one-hot encoded (197 columns in total). Categories that appear only in a validation fold are encoded as all zeros.
- **Linear models:** two changes to the 24 features. The full flight code (e.g. `EY 353`) is added as a one-hot feature, which raised R^2 from 0.796 to 0.896 in early linear tests (it made XGBoost worse, so the trees leave it out). `flight_num` is dropped, because a flight number is a label and a linear coefficient on it has no meaning. `duration` and `days_left` are expanded into cubic splines (6 knots), so the model can fit curved effects, and all numeric columns are standardised. This gives 1,280 columns.

5 Cross-Validation Framework

Instead of a fixed validation set, every model used the same 5-fold `KFold` split (shuffled, seed 42) over all 15,589 training rows:

- each model is trained on 4 folds and predicts the 5th, five times, so every training row gets an **out-of-fold (OOF)** prediction from a model that never saw it;
- the test prediction is the average of the five fold models;
- all predictions are stored as prices in rupees, so every model is scored the same way and models can be averaged;
- the blend (Section 8.1) and the lookup (Section 8.2) reuse the same folds, so no step ever sees a row's own price.

Each tree model has two settings:

- **Target:** *log* trains on log(1 + price); *raw* trains on the price itself.
- **Training:** *pooled* fits one model on both classes; *per-class* fits separate Business and Economy models inside each fold.

All linear models use a log target and per-class training. Their log predictions are also capped to the training fold's range ±0.1, because a linear model can predict absurd prices for rare combinations (a tree cannot).

Four metrics are reported: RMSE, MAE and R^2 on the price, and RMSLE (RMSE of log prices). RMSE is used for every selection decision.

6 Models Tried

We trained eleven models in two families, building each family up step by step: every new model was chosen to fix a specific weakness of the one before it. The first bullet of each model gives the reason for that step. The four linear models serve as a point of comparison; the final prediction comes from the tree-based models only. All scores are 5-fold CV scores on out-of-fold predictions over all 15,589 training rows, collected in Table 4.

6.1 Linear Models

Why: linear models show how much of the price can be explained by adding up one effect per feature, and set a reference point for the trees. All four use the linear pipeline of Section 4, a log target and separate models for Business and Economy.

6.1.1 Linear Regression (Baseline)

- **Why:** the simplest linear model, with nothing added, so every later change can be measured against it.
- Ordinary least squares on log(price), with no penalty on the coefficients.
- With 1,280 one-hot and spline columns, many of them flights seen only a few times, an unpenalised fit can memorise noise in rare categories.

6.1.2 Ridge (L2 Penalty)

- **Why:** to stop rare flights and airlines from getting extreme coefficients, the main risk of the unpenalised baseline.
- Adds an L2 penalty $\alpha \sum \beta^2$; α tuned over {0.03, 0.1, 0.3, 1, 3}, best $\alpha = 0.3$.
- RMSE fell by only 28 (11,262 → 11,234), so unstable coefficients were not the main problem.

6.1.3 Lasso (L1 Penalty)

- **Why:** to test whether many of the 1,280 columns are useless; an L1 penalty can remove them.
- Uses $\alpha \sum |\beta|$, which sets many coefficients exactly to zero and so also selects features. $\alpha = 3 \times 10^{-5}$, the best of {3e-5, 1e-4, 3e-4, 1e-3}.
- Keeps 520 of 699 coefficients for Business and 645 of 1,150 for Economy.
- RMSE 11,256, no better than Ridge. Removing columns did not help, so the limit is not the number of features but the form of the model.

6.1.4 Ridge with Airline-Specific Duration Slopes

- **Why:** an additive model gives every airline the same duration effect, but most of the error sits in long one-stop Business trips on premium airlines (Section 9).
- Ridge plus one extra column per airline with at least 30 training rows in the class: `duration / 10` on that airline's rows and 0 elsewhere, so each airline gets its own duration slope.
- α tuned over {0.1, 0.3, 1, 3}; best $\alpha = 0.3$.
- RMSE 11,018, a gain of 216 over Ridge; the best linear model.

Why we moved to trees. The one interaction we built by hand gained eight times more than any penalty. The data clearly has many more such interactions (airline × stops × duration × class), and building each one by hand is not practical. Trees find these interactions on their own.

6.2 Tree-Based Models

6.2.1 Decision Tree (Baseline)

- **Why:** the simplest tree; it tests whether learning interactions automatically beats the linear models.
- A single regression tree on log(price), pooled, with `max_depth` = 14 and `min_samples_leaf` = 3 to limit its variance.
- RMSE 11,797, worse than every linear model. A single deep tree has high variance: small changes in the training rows change its splits, and its leaves fit the noise in the fares. Not used in the blend.

6.2.2 Random Forest (Log Target)

- **Why:** averaging many different trees cancels much of a single tree's variance.
- 500 trees, each grown on a bootstrap sample and trying only 80% of the one-hot columns at each split, so the trees differ and their errors partly cancel; `min_samples_leaf` = 2; log target, pooled.
- RMSE 10,256, a gain of 1,541 over the single tree and better than every linear model; the lowest MAE of any single model.
- The trees are grown independently, so no tree corrects the errors of the others: averaging lowers variance but not the shared bias.

6.2.3 Random Forest (Raw Price)

- **Why:** a log model predicts a slightly low, median-like fare, while RMSE rewards the mean; training on the price itself predicts the mean directly.
- The same forest trained on the price in rupees, with separate forests per class.
- Slightly worse alone (RMSE 10,389), but it makes different errors from the log forest, so it was kept for the blend.

6.2.4 XGBoost (Log Target)

- **Why:** boosting fixes the forest's weakness. It builds trees one after another, each fitting the errors the previous trees left, so it also reduces bias.
- 1,000 trees, learning rate 0.03, depth 5, L2 penalty 5, minimum child weight 2, 80% row and 70% column sampling, categorical columns handled directly; log target, pooled.
- The small learning rate, L2 penalty and sampling keep it from fitting noise; shallow trees (depth 5) clearly beat deeper ones (Section 7).
- RMSE 10,127, a further gain of 129 over the forest; the best single model.

6.2.5 XGBoost (Gamma Objective)

- **Why:** the Gamma loss treats errors as relative, like a log target, but predicts the mean fare, so it avoids the low bias of the log model.
- The same shallow XGBoost with 800 trees and the Gamma loss on the raw price, per class.
- Slightly worse alone (RMSE 10,215), but a different error pattern for the blend.

6.2.6 LightGBM (Tweedie Objective)

- **Why:** a second boosting library that builds trees differently makes different errors. XGBoost grows each tree level by level, while LightGBM always splits the leaf that reduces the loss most, and it smooths rare categories (`cat_smooth`).
- 1,500 trees, learning rate 0.03, only 7 leaves per tree, at least 5 rows per leaf, L2 penalty 2, `cat_smooth` 20, 80% row and 70% column sampling.
- Tweedie loss (power 1.5) on the raw price, per class; like Gamma, it predicts the mean with relative errors.
- RMSE 10,174, the second-best single model.

6.2.7 LightGBM (Log Target)

- **Why:** to see how the same algorithm does with a log target, and to add one more distinct model to the blend.
- The same LightGBM set-up with 15 leaves per tree, log target, per class.
- The lowest RMSLE of any single model (0.1288).

Why we moved to a blend. The six models from Random Forest onwards are within 262 RMSE of each other, no single target or training scheme was best for every algorithm, and models with different targets make different errors. Averaging them (Section 8.1) uses this, instead of choosing one.

6.3 Choice of Target

A log target makes errors relative and handles the skew, but its back-transformed prediction is slightly below the mean; for XGBoost (log) the best correction factor is 1.016 for Business and 1.018 for Economy, hence the 1.01 factor in the final blend. Raw, Tweedie and Gamma models predict the mean directly. The log target won for Random Forest (10,256 vs 10,389) and XGBoost (10,127 vs 10,215 with Gamma); Tweedie won for LightGBM (10,174 vs 10,235).

Table 4: Models tried and their best 5-fold CV scores (RMSE and MAE in rupees). Best value per column in bold. Blend: used in the final blend.

Model	Key settings	RMSE	MAE	R^2	RMSLE	Blend
Linear Regression	Log, per class	11,262	4,126	0.9171	0.1791	no
Ridge	$\alpha = 0.3$, per class	11,234	4,090	0.9176	0.1744	no
Lasso	$\alpha = 3 \times 10^{-5}$, per class	11,256	4,102	0.9172	0.1738	no
Ridge + slopes	$\alpha = 0.3$, slope per airline	11,018	4,039	0.9207	0.1737	no
Decision Tree	Log, pooled, depth 14	11,797	3,782	0.9091	0.1597	no
Random Forest (log)	500 trees, pooled	10,256	**3,209**	0.9313	0.1332	yes
Random Forest (raw)	500 trees, per class	10,389	3,321	0.9295	0.1367	yes
XGBoost (log)	1,000 trees, depth 5, pooled	**10,127**	3,273	**0.9330**	0.1305	yes
XGBoost (Gamma)	800 trees, depth 5, per class	10,215	3,277	0.9318	0.1293	yes
LightGBM (Tweedie)	1,500 trees, 7 leaves, per class	10,174	3,494	0.9324	0.1367	yes
LightGBM (log)	1,500 trees, 15 leaves, per class	10,235	3,275	0.9316	**0.1288**	yes

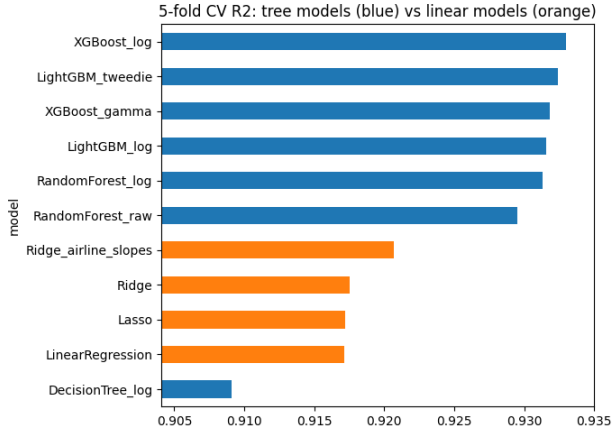


Figure 3: 5-fold CV R^2 of every model: tree-based models in blue, linear models in orange.

7 Hyperparameter Tuning

All tuning used the same 5-fold CV and selected on RMSE (Table 5).

Table 5: Hyperparameter tuning by model, with the CV RMSE of each option.

Model	Settings explored	Selected
Ridge	$\alpha \in \{0.03, 0.1, 0.3, 1, 3\}$: 11,257 / 11,245 / 11,234 / 11,252 / 11,352	$\alpha = 0.3$
Lasso	$\alpha \in \{3e\text{-}5, 1e\text{-}4, 3e\text{-}4, 1e\text{-}3\}$: 11,256 / 11,369 / 11,683 / 12,487	$\alpha = 3 \times 10^{-5}$
Ridge + slopes	$\alpha \in \{0.1, 0.3, 1, 3\}$: 11,028 / 11,018 / 11,031 / 11,139	$\alpha = 0.3$
Decision Tree	Pooled vs per class: 11,797 vs 12,008; depth and leaf size fixed	Pooled
Random Forest	Pooled vs per class: log 10,255 vs 10,275, raw 10,399 vs 10,389; other settings fixed	Log pooled, raw per class
XGBoost	Depth 7, L2 = 2 vs depth 5, L2 = 5: log 10,394 vs 10,128, Gamma 10,410 vs 10,214. Pooled vs per class: log 10,128 vs 10,268, Gamma 10,217 vs 10,214	Depth 5, L2 = 5; log pooled, Gamma per class
LightGBM	Log: 63 vs 15 leaves, 10,359 vs 10,235. Tweedie: 63 vs 7 leaves, 10,350 vs 10,175. Pooled vs per class: log 10,259 vs 10,235, Tweedie 10,263 vs 10,175	15 leaves (log), 7 leaves (Tweedie), per class
Blend and lookup	See Tables 6 and 7	Average × 1.01; weight 0.9

In all four boosting variants the shallower set-up won, by 124–266 RMSE, and per-class training helped four of the seven tree models. Ideas tried and dropped because they did not beat the final set-up: the full flight code as a tree feature, average-price features of similar flights, dropping high-cardinality categories, stronger smoothing of categories, finer histogram bins and Huber loss.

8 Final Blend and Exact-Match Lookup

8.1 Blend of Six Tree Models

The six tree models other than the Decision Tree are averaged with equal weights. No weights are fitted, so the blend cannot overfit the folds.

Table 6: Blend options (5-fold CV). The option with the lowest RMSE was used.

Option	RMSE	MAE	R^2	RMSLE
Best single model (XGBoost, log)	10,127.0	3,273.2	0.9330	0.1305
Average of 6 models	10,005.5	**3,224.1**	0.9346	**0.1280**
Average of 6 models × 1.01	**9,990.3**	3,299.2	**0.9348**	0.1287

Averaging lowers RMSE by 122 over the best single model, and the 1.01 correction by a further 15. The correction slightly raises MAE and RMSLE, because it moves predictions towards the mean, which RMSE rewards and MAE does not.

8.2 Exact-Match Lookup

Some test rows have an **exact twin** in train: the same flight, source, destination, class, days left, departure time, arrival time and stops. For these rows the twin's price is often a better guess than any model, so the prediction mixes the twins' mean price with the blend:

$$\hat{y} = w \cdot \bar{y}_{\text{twin}} + (1 - w) \cdot \hat{y}_{\text{blend}}.$$

In each CV fold the lookup table is built from the training folds only, so a row never sees its own price. Under this rule 7.1% of training rows have a twin; in the test set, using all training rows, 8.3% do.

Table 7: Lookup weight sweep (5-fold CV, on top of the blend × 1.01).

Weight w	RMSE	MAE	R^2
0.00	9,990.3	3,299.2	0.9348
0.50	9,899.8	3,222.4	0.9360
0.70	9,883.0	3,194.1	0.9362
0.90	**9,877.4**	3,167.4	**0.9363**
1.00	9,878.7	**3,154.5**	0.9363

Weights 0.60, 0.75, 0.80 and 0.85 were also tested (RMSE 9,890.0, 9,880.5, 9,878.8 and 9,877.7).

The best weight, $w = 0.9$, almost fully trusts the twin's price. The final CV score is **RMSE 9,877.4, MAE 3,167.4,** R^2 **0.9363**.

Test predictions. Each model's test prediction is the average of its five fold models. The blend is multiplied by 1.01, and the lookup (weight 0.9) is applied to the 8.3% of test rows with a twin. Predictions are clipped at a minimum of Rs. 3,000 (lowest training price: Rs. 3,801), and the file was checked against the sample submission for shape (3,898 × 2), columns, id order and missing values.

9 Results and Discussion

9.1 Model Families Compared

Every tree model except the single Decision Tree beats the best linear model (RMSE 10,127–10,389 against 11,018). The three penalties gave nearly the same linear score (R^2 0.9171–0.9176), while each move in the tree family (forest, boosting, blend) gave a clear gain: 1,541, 129 and 137 RMSE. The ranking matches the reasons in Section 6: the gains came from capturing interactions and from lowering variance and bias, not from penalties.

9.2 Almost All Remaining Error Is in Expensive Business Fares

Table 8: Error of the final model (CV) by true price. SSE share = share of the total squared error. MAPE = mean absolute percentage error.

Price band (Rs.)	Rows	Row share	RMSE	MAPE	SSE share
< 8k	4,971	31.9%	603	5.4%	0.1%
8k–15k	3,795	24.3%	1,154	7.1%	0.3%
15k–40k	3,697	23.7%	3,423	7.8%	2.8%
40k–100k	1,992	12.8%	13,544	11.8%	24.0%
> 100k	1,134	7.3%	31,220	12.1%	72.7%

- Business tickets cause 98.5% of the squared error and Economy only 1.5%.
- Tickets above Rs. 100k are 7.3% of rows but cause 72.7% of the squared error; the worst 5% of rows cause 92%.
- Within Business, one-stop trips cause 91.1% of all squared error (RMSE 19,915), non-stop flights only 6.2% (RMSE 5,789).
- Three Business airlines cause 61% of all squared error: Nakatomi Airlines (43.7%, RMSE 48,533 on only 282 rows), Stark Aviation (9.5%) and Wayfarer Aviation (7.7%, spread over 2,496 rows with RMSE 6,832).

In relative terms the model is accurate everywhere: the mean percentage error is 5–8% below Rs. 40k and about 12% above it.

9.3 What the Models Rely On

Figure 4 ranks features by how much they reduce the training loss in a LightGBM model fitted on all training rows. The airline × class combination comes first, followed by the class itself: class and airline set the price level, and the itinerary adjusts it.

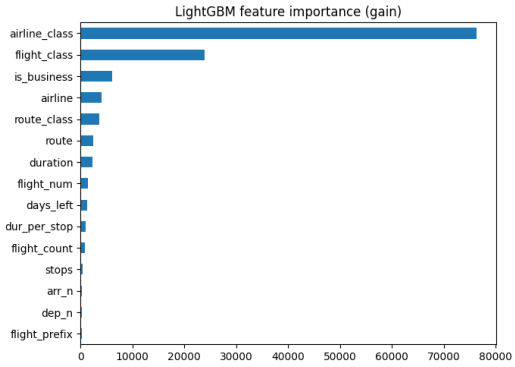


Figure 4: LightGBM feature importance (gain), top 15 features.

10 Best Model

Final Model

An **equal-weight average of six tree models, multiplied by 1.01 and combined with an exact-match lookup**.

- **Base models:** Random Forest (log, pooled), Random Forest (raw, per class), XGBoost (log, pooled), XGBoost (Gamma, per class), LightGBM (Tweedie, per class), LightGBM (log, per class), with the settings in Table 4.
- **Inputs:** the 24 features of Section 4; `id` excluded.
- **Lookup:** for test rows with an exact twin in train, 0.9 × the twins' mean price + 0.1 × the blend.
- **Training data:** each model's test prediction averages its five fold models (each trained on 80% of the 15,589 rows); the lookup table uses all training rows.
- **Score:** 5-fold CV RMSE 9,877.4, MAE 3,167.4, R^2 0.9363, against RMSE 10,127.0 for the best single model.

Why this model. Each of its three parts lowered CV RMSE on the same folds: averaging (10,127 → 10,006), the 1.01 correction (→ 9,990) and the lookup (→ 9,877). Only two numbers (the 1.01 factor and the lookup weight) were tuned on the CV predictions, so the risk of overfitting the folds is low.

11 Key Insights and Limitations

Key insights

1. The data contained its own fixes: the flight prefix recovered every missing airline, and 97.8% of missing durations came from the same flight, route and stops.
2. Business and Economy behave like two separate markets; airline × class is the most important feature.
3. Trees beat linear models because they learn interactions on their own, and shallow boosted trees beat deep ones on these noisy fares.
4. Blending, the 1.01 correction and the exact-match lookup each gave a small, consistent gain.

Limitations.

- All results come from one 5-fold split, and the 1.01 factor and lookup weight were chosen on the same OOF predictions, so the final CV score is slightly optimistic.
- The Decision Tree and Random Forest settings were fixed rather than searched.
- Large price differences for the same flight limit the accuracy any model can reach.

12 Conclusion

We built a complete pipeline for flight price regression, from EDA and feature engineering through eleven models to an average of six tree models with an exact-match lookup. The final model reached CV R^2 0.9363 (RMSE 9,877). Almost all remaining error comes from expensive Business fares whose price varies widely for the same flight, so further gains are more likely to come from new information, such as the actual departure date, than from more complex models.

PART II

REPORT: Multiclass Classification

Vehicle Type Classification

Girish Chougule (MT2026028) | Yashik Jain (MT2026117)

1 Introduction

This part predicts the type of a vehicle (Sedan, SUV or Sports Car) from 20 numeric measurements of its engine, body, performance and fuel efficiency. The task is the VehicleVision Kaggle competition, scored by accuracy.

The competition adds one twist: the hidden test data was measured in a different environment and contains extra random measurement error. A model that is accurate on clean training data can still lose accuracy on the leaderboard, so robustness to noise was treated as being as important as clean accuracy.

The work covers EDA and one fixed train/validation split; preprocessing, including removal of hidden duplicate rows; three model families (logistic regression, QDA, decision tree), each taken from a baseline to tuned and noise-trained versions, followed by XGBoost and two blends of logistic regression and XGBoost; robustness checks with noise added to the validation data; and a comparison across cross-validation, validation and the public Kaggle leaderboard.

At a glance

Data:	7,500 training rows after removing duplicates, 20 numeric features, 3 classes
Models tried:	9 (three logistic regression, two QDA, one decision tree, one XGBoost, two LR + XGBoost blends)
Best public score:	0.907 (LR + XGBoost blend, 50 : 50)
Final model:	LR + XGBoost blend (final notebook: 52 : 48 variant, public score 0.905)

2 Problem and Dataset

Target. `vehicle_class` has three classes: 0 = Sedan (33.9%), 1 = SUV (33.3%) and 2 = Sports Car (32.9%). Always predicting the most common class gives only about 34% accuracy, the floor every model must beat.

Inputs. All 20 features are numeric (`float64`), so no categorical encoding is needed. `vehicle_id` is a record number and is not used.

3 Exploratory Data Analysis (EDA)

The EDA checks data quality, how well the classes can be separated, and whether the test data is really noisier than the training data. It shows a clean dataset with one main difficulty: Sedans overlap the other two classes.

Table 9: Input features.

Group	Features
Engine and power	engine displacement (l), horsepower (hp), torque (N·m), cylinders (integer 3–10), battery capacity (kWh)
Size and body	weight (kg), wheelbase, length, width, height (mm), fuel-tank capacity (l), cargo capacity (l), ground clearance (mm)
Performance	0–100 km/h time (s), braking distance (m), turning radius (m), tire width (mm), drag coefficient
Efficiency	city and highway fuel efficiency (km/l)

3.1 Class Balance, Missing Values and Duplicates

- **Class balance:** the 10,000 training rows split into 3,389 Sedans (33.9%), 3,326 SUVs (33.3%) and 3,285 Sports Cars (32.9%) (Figure 5, left). Accuracy is therefore a fair metric, no re-sampling is needed, and always predicting the largest class scores only about 34%.
- **Missing values:** none in the training or the test file.
- **Duplicates:** 2,500 rows (25%) repeat another row's 20 features and label. Every duplicate group has exactly two rows and no group has conflicting labels. Because `vehicle_id` is unique even for the copies, a plain duplicate check finds none; the copies only show up when the 20 feature columns are compared.
- **Where the copies are:** all 2,500 copies sit in one block at the end of the file (`vehicle_id` 7,501–10,000), and their class mix matches the whole dataset (Figure 5, right). They are re-sampled rows that add no new information, so they were removed before any split (7,500 unique rows remain). The class mix is also the same across all `vehicle_id` ranges, so the id carries no class information and is not used as a feature.

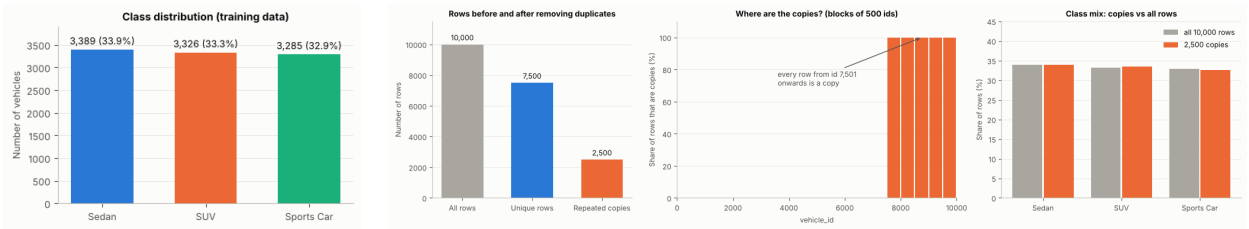


Figure 5: Class distribution of the 10,000 training rows (left); rows before and after removing duplicates, position of the copies in the file, and their class mix (right).

3.2 How Well Can the Classes Be Separated?

- **One feature at a time:** a depth-3 tree trained on a single feature (5-fold CV) reaches at most 68.9% (braking distance), followed by battery capacity (64.9%) and drag coefficient (63.3%). The two fuel-efficiency features are the weakest (41–44%), barely above the 34% floor (Figure 6, left). Mutual information and the ANOVA F-statistic rank the features almost identically (rank correlation 0.97–0.99), so good accuracy requires combining features.
- **All features together:** two principal components explain 79.5% of the variance and five explain 90%, so the 20 features describe a few underlying properties. Even without labels, the three classes form visible clouds. The LDA projection, which looks for the directions that best separate the classes, puts SUVs and Sports Cars at opposite ends, while Sedans stay in the middle and overlap both (Figure 6, right).

- **Implication:** a plain linear classifier on all 20 features reaches 89.2% CV accuracy. Linear models are therefore a strong starting point, and the overlap around Sedans sets a practical ceiling of about 90% on clean data.

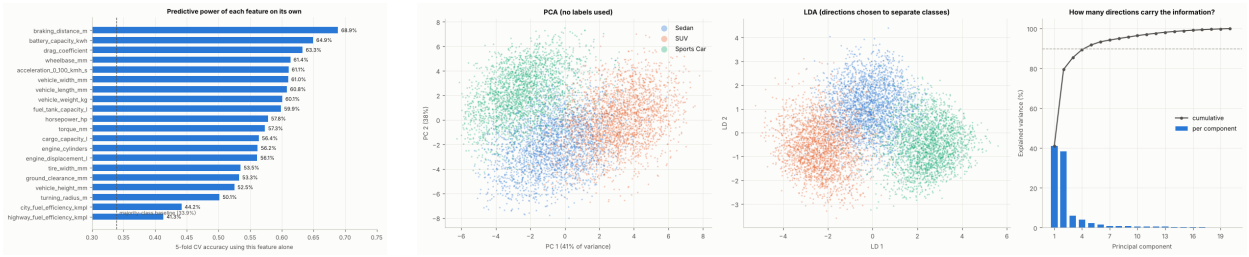


Figure 6: 5-fold CV accuracy of a depth-3 tree on each feature alone; the dashed line is the 34% majority-class floor (left). Two-dimensional PCA and LDA views of all 20 features, and the variance explained by each principal component (right).

3.3 Capped Values, Outliers and Test Noise

- **Capped values:** many features pile up exactly at their minimum or maximum, because the data was clipped to fixed realistic ranges; about a quarter of rows sit at the 110 mm floor or 300 mm ceiling of ground clearance. These caps are class-specific (Figure 7, left): Sports Cars make up about 75% of rows at the 110 mm clearance floor and over 80% at the 150 l cargo floor, and almost every vehicle at the 2,050 mm width ceiling is an SUV. They carry class information and were kept.
- **Outliers:** only 135 of 7,500 rows (1.8%) contain an IQR outlier (52 have $|z| > 3$), mostly in fuel efficiency and turning radius, and all values are physically plausible. They were kept: the test data is noisier than the training data, and removing extreme training rows would leave models less prepared for unusual test inputs.
- **Train vs test spread:** test means match training means, so the test vehicles come from the same population. The test spread, however, is larger on seven engine and performance features (displacement, highway efficiency, cylinders, horsepower, torque, tire width, battery), with test-to-train SD ratios of 1.02–1.04, while the size features do not change. With a standard error of about 0.016 per ratio, only displacement is clearly beyond two standard errors, but the consistent pattern points to added noise of roughly 0.2–0.25 training SD on those features.
- **Effect of that noise:** adding 0.25 SD of noise to the seven features visibly blurs the class clouds on horsepower and battery capacity (Figure 7, right). Yet a linear model on all 20 features loses only 0.2 points (89.4% → 89.2% on the same vehicles), because the size features, which carry little extra noise, still identify most vehicles.
- **Implication:** every model was also evaluated on noise-injected validation data over a range of levels, noise augmentation was tried, and models that combine many features were preferred over models that rely on a few sharp thresholds on the noisy engine features.



Figure 7: Class mix of the rows sitting at the most common caps (left). Horsepower vs battery capacity before and after adding the estimated test noise (right).

4 Preprocessing

The shared data was changed in only two ways: duplicates were removed and one split was fixed. Every other transformation is learned inside each model's pipeline on training data only, so no validation information leaks into any model. The original CSV was never modified; the cleaned data was saved as a separate file.

4.1 Shared Steps (Used by Every Model)

- **Missing values:** none in any column, so no filling was needed.
- **Duplicate removal:** after checking that every duplicate group has one label, the 2,500 repeated rows were dropped (keeping the first copy), so 10,000 rows became 7,500. Class shares stayed almost unchanged (33.85 / 33.20 / 32.95%). This is done *before* any split, so copies of one vehicle cannot end up in both training and validation folds.
- **Validity checks:** data types, labels (only {0, 1, 2}), and infinite, negative, zero and constant values were checked. No problems were found.
- **Outliers and capped values:** kept unchanged (Section 3). All 20 features are used.

4.2 Model-Specific Steps

- **Logistic regression:** a `StandardScaler` inside the pipeline, fitted on training folds only. Feature scales differ by four orders of magnitude, and the L2 penalty would otherwise shrink coefficients unevenly.
- **QDA:** no scaling. QDA fits a separate mean and full covariance per class, so its decisions do not depend on feature scale.
- **Decision tree:** each correlated block is replaced by the mean of its standardised members: a size score (weight, wheelbase, length, width, fuel tank), a power score (horsepower, torque, displacement, cylinders, tire width), a height score (height, ground clearance, cargo) and an efficiency score (city and highway efficiency). With the five other features the tree sees 9 inputs. Averaging k features with independent noise divides the noise standard deviation by $\sqrt{k}$, so the tree's split points are more stable.
- **XGBoost and blends:** no scaling for XGBoost. Its trees split on thresholds, and rescaling a feature does not change the order of its values, so the model is unchanged. In the blends, LR keeps its own `StandardScaler` pipeline and XGBoost uses the raw features; only their predicted class probabilities are combined.

4.3 Noise Simulation and Noise Training

To imitate the test data, we added Gaussian noise with standard deviation = (noise level) × (the feature's training standard deviation), independently to each feature. Labels were never changed.

- **For evaluation:** validation rows were perturbed at levels 0.1 and 0.2. The tree was also tested at 0.3, and with noise on the seven engine features only.
- **For training (augmentation):** training rows were perturbed instead, at level 0.1 for LR and QDA; the tree used two extra noisy copies at level 0.2. The noise scale came from the training fold only, and test features were never modified.

5 Train–Validation Split

We used one stratified 80/20 split of the 7,500 rows (seed 42): 6,000 training rows (class shares 33.8 / 33.2 / 33.0%) and 1,500 validation rows (33.9 / 33.2 / 32.9%). The split was saved to `vehicle_split_indices.csv` and reloaded by every notebook, so differences between models come from the models, not from different splits. All tuning used 5-fold stratified CV on the 6,000 training rows; the 1,500 validation rows were used only for evaluation. Final models were retrained on all 7,500 rows before predicting the test set.

6 Models Tried

We trained nine models. Six belong to three families (logistic regression, QDA and decision tree), each taken from a baseline to tuned and noise-trained versions; they were followed by XGBoost and two blends of logistic regression and XGBoost. As in Part I, each step was chosen to address a weakness of the previous one, and the first bullet of each model gives the reason. CV means 5-fold stratified CV on the 6,000 training rows, and validation means the 1,500 held-out rows; all scores are collected in Table 10.

6.1 Logistic Regression

Why: the EDA showed near-linear class boundaries, so a linear classifier was the natural first family.

6.1.1 Logistic Regression (Baseline)

- **Why:** the simplest linear classifier, and the reference for every other model.
- Pipeline: `StandardScaler` → `LogisticRegression` ($C = 1.0$, L2, `max_iter` = 2000, `random_state` = 42).
- CV fold scores ranged from 0.8875 to 0.9025. Per-class validation F1: Sedan 0.83, SUV 0.90, Sports Car 0.92.

6.1.2 Logistic Regression (Tuned, $C = 0.1$)

- **Why:** the features form strongly correlated blocks (14 pairs above $|r| = 0.9$), which can make the coefficients unstable; a stronger penalty might fix this.
- C (the inverse penalty strength) tuned over $\{0.001, 0.01, 0.1, 1, 10, 100\}$ by 5-fold grid search on macro F1.
- Best $C = 0.1$ (CV macro F1 0.8969), only 0.1 points above $C = 1.0$. A very strong penalty underfit ($C = 0.001$: 0.8677), and $C \geq 1$ plateaued at 0.8945–0.8955. The penalty was not the limit.

6.1.3 Logistic Regression (Noise-Trained)

- **Why:** the test data has extra measurement noise; training on noisy copies shows the model what noisy inputs look like and makes it less sensitive to them.
- The baseline pipeline ($C = 1.0$) trained on a copy of the training features with 0.1 SD Gaussian noise added; labels unchanged. The noise scale was computed inside each fold.
- CV accuracy within one standard deviation of the baseline, but the highest validation accuracy of all nine models.

Why we moved to QDA. All three LR variants scored about the same, so the remaining errors are not caused by the penalty or by noise. LR draws one straight boundary between each pair of classes and assumes the classes differ only in their position. In the data, classes also differ in spread and in how features move together, and almost all errors involve Sedan, which lies between the other two classes. QDA tests whether curved boundaries that use these class differences separate Sedan better.

6.2 QDA

6.2.1 QDA (Baseline)

- **Why:** QDA fits a separate mean and full covariance matrix per class, so its boundaries are curved and it uses the correlations between features. It was chosen over Gaussian Naive Bayes, which assumes independent features and would ignore the correlated blocks.
- Per-class mean vectors, full covariance matrices and class priors (0.3385 / 0.332 / 0.3295).
- `reg_param` tuned over 0–0.5 by 5-fold CV; macro F1 fell steadily from 0.8895 at 0 to 0.8867 at 0.5, so `reg_param` = 0 was kept.
- 0.7 CV points below LR; per-class validation F1: Sedan 0.82, SUV 0.89, Sports Car 0.92. Curved boundaries did not separate Sedan better.
- The most noise-sensitive family: validation accuracy fell from 0.8747 to 0.8500 at noise 0.2.

6.2.2 QDA (Noise-Trained)

- **Why:** QDA was the model most hurt by noise, so it had the most to gain from noise training.
- QDA (`reg_param` = 0) trained on features with 0.1 SD Gaussian noise.
- Validation accuracy rose from 0.8747 to 0.8780, still below LR.

Why we moved to a decision tree. LR and QDA both draw one smooth boundary that uses all features at once. The EDA showed a different kind of signal: many features pile up at fixed, class-specific limits (for example, about 75% of rows at the 110 mm ground-clearance floor are sports cars). A tree splits on thresholds, so it tests whether such rules help. Trees react strongly to small input changes, so every tree setting was chosen on CV accuracy *at noise 0.2*, not on clean accuracy; when settings tied within one standard error, the simpler tree was chosen.

6.3 Decision Tree

6.3.1 Unconstrained Tree

- **Why:** to see how a tree behaves with no limits.
- Default settings grew a tree of depth 24 with 519 leaves (about 12 training rows per leaf).
- 100% training accuracy against 82.3% CV accuracy, a clear case of overfitting. At noise 0.3, CV accuracy fell to 78.2%; validation accuracy was 0.8140.

6.3.2 Depth and Leaf Tuning

- **Why:** limiting depth and leaf size stops the tree from memorising single rows.
- Depth curve (1–20): CV accuracy peaked at depth 5 (0.8668 clean, 0.8526 at noise 0.2) and fell for deeper trees.
- Grid over depth 4–8, minimum leaf size 1–80 and gini/entropy (60 settings): the best was depth 7, 40 rows per leaf, entropy (0.8569 ± 0.0077 at noise 0.2), with 24 settings statistically tied.
- Cost-complexity pruning gave no gain once depth and leaf size were limited.

6.3.3 Final Tree

- **Why:** the tuned tree's split points still moved with noise. Averaging each correlated block into one score divides independent noise by $\sqrt{k}$ (Section 4), and training on noisy copies teaches the tree splits that hold under noise.
- Block-averaged inputs, depth 7, at least 40 rows per leaf, entropy, trained with two extra noisy copies of the data at 0.2 SD; 88 leaves.
- Averaged over three CV splits (seeds 42, 7, 123), noise training beat the same tree without it: clean 0.8666 vs 0.8611, noise 0.2 0.8594 vs 0.8558.
- Predicting each test row by majority vote over 15 noisy copies added only 0.15 points while making prediction 15 times slower, so it was not used.
- Shuffling one feature on noisy validation data and measuring the accuracy drop: acceleration costs 21.5 points and battery capacity 4.6 points, while single size features cost only 1.2–1.6 points each, because the tree uses their average.
- Still about 3 CV points behind LR.

6.4 Why LDA Was Not Included

Linear discriminant analysis (LDA) was considered but not trained. LDA is QDA with one covariance matrix shared by all classes, so QDA already covers it, and its boundaries are linear, which LR already learns directly without assuming Gaussian features. LDA would therefore add no new kind of boundary.

Why we moved to XGBoost. A single tree, even with stable inputs, was about 3 points behind LR. Boosting keeps what trees do well (threshold rules and interactions between features) but builds many shallow trees in sequence, each correcting the errors of the previous ones, so no single unstable tree decides the answer. XGBoost is also a natural partner for LR in a blend, because its errors come from a different kind of model.

6.5 XGBoost

- **Why:** to test whether many boosted shallow trees close the gap between the single tree and LR.
- Settings (fixed from the team's tuning experiments): 290 trees; `max_depth` = 5; `learning_rate` = 0.042; `min_child_weight` = 6; `subsample` = 0.8; `colsample_bytree` = 0.8; `reg_alpha` = 0.4 (L1); `reg_lambda` = 8 (L2); `tree_method` = `hist`; `random_state` = 42.
- Training each tree on 80% of the rows and 80% of the features, plus the L1 and L2 penalties, keeps the 290 trees from overfitting.

- CV accuracy 0.8877, 2.1 points above the single tree but 0.8 below LR. Per-class validation F1: Sedan 0.81, SUV 0.89, Sports Car 0.91. Validation accuracy at noise 0.2: 0.8638 (10 noise draws).

Why we moved to blends. XGBoost closed most of the tree's gap but did not beat LR. The two models work in different ways: LR draws smooth straight boundaries across all features, while XGBoost learns local threshold rules. Averaging their predicted probabilities can fix vehicles on which only one of the two is wrong. For each vehicle, the blend computes

$$P_{\text{blend}}(c) \;=\; w\,P_{\text{LR}}(c) + (1-w)\,P_{\text{XGB}}(c), \qquad \hat{y} = \arg\max_{c} P_{\text{blend}}(c),$$

using the LR baseline pipeline (`max_iter` = 1600) and the XGBoost model above. This is called soft voting. The weight w was fixed before predicting the test set.

6.6 Soft-Voting Blends (LR + XGBoost)

6.6.1 LR + XGBoost (50 : 50)

- **Why:** with no evidence yet that either model deserves more weight, equal weights are the neutral starting point.
- $w = 0.5$.
- Validation accuracy at noise 0.2: 0.8696.
- The best public score of all submissions, and first place on the public leaderboard when it was submitted.

6.6.2 LR + XGBoost (52 : 48, Final Notebook)

- **Why:** LR was the stronger single model on the leaderboard (0.904 vs 0.901), so it receives a slightly larger share.
- $w = 0.52$. Moving 2% of the weight only changes vehicles on which the two models almost tie: 2 of the 2,500 test predictions, compared with the 50 : 50 blend trained with the same library versions.
- Validation accuracy at noise 0.2: 0.8695 ± 0.0042.
- Library versions are pinned (scikit-learn 1.9.1, XGBoost 3.4.1), and the final notebook checks that it regenerates the submitted file row for row.

6.7 Summary of Models

Table 10 records each model's settings and best scores.

Kaggle submissions. Each of the nine final pipelines was retrained on all 7,500 rows and predicted the 2,500 unmodified test rows. Before upload, every file was checked for 2,500 rows, the columns `vehicle_id` and `vehicle_class`, ids in test-file order, labels only in {0, 1, 2} and no missing values. The predicted class mix was balanced in every file (about 31–34% per class). The leaderboard ranks the LR, QDA and tree families in the same order as CV, but it places XGBoost and the two blends higher than CV and validation do: on the leaderboard the blends come first, while on CV and validation LR alone is slightly ahead. These leaderboard gaps are only a few thousandths, that is, a handful of test rows.

7 Results and Discussion

7.1 Linear Models Lead on CV and Validation; the Blend Leads on Kaggle

Table 10: Models tried and their best accuracy. CV = 5-fold stratified CV on the 6,000 training rows; Validation = the 1,500 held-out rows; Kaggle = public leaderboard. Best value per column in bold.

Model	Key settings	CV	Validation	Kaggle
LR baseline	StandardScaler + LR, $C = 1.0$, L2	0.8955	0.8840	0.904
LR tuned	$C = 0.1$, chosen by 5-fold grid search on macro F1	**0.8965**	0.8827	0.902
LR noise-trained	Baseline pipeline trained on features + 0.1 SD noise	0.8960	**0.8860**	0.903
QDA baseline	`reg_param` = 0, chosen by 5-fold CV over 0–0.5	0.8888	0.8747	0.886
QDA noise-trained	`reg_param` = 0, trained on features + 0.1 SD noise	0.8864[a]	0.8780	0.893
Decision tree	9 block-averaged inputs, depth 7, ≥40 rows per leaf, entropy, 2 noisy copies at 0.2 SD	0.8666[b]	0.8487	0.868
XGBoost	290 trees, depth 5, learning rate 0.042, row/feature sampling 0.8, L1 0.4, L2 8	0.8877[c]	0.8713[c]	0.901
LR + XGBoost 50:50	Soft voting, equal weights	0.8898[c]	0.8800[c]	**0.907**
LR + XGBoost 52:48	Soft voting, LR weight 0.52 (final notebook)	0.8903[c]	0.8800[c]	0.905

[a] 5-fold CV over all 7,500 rows (the clean-trained QDA scored 0.8848 on the same folds). [b] Mean of three CV splits (seeds 42, 7, 123). [c] Computed with scikit-learn 1.9.1 and XGBoost 3.4.1.

The ranking LR > QDA > tree holds in CV, on validation and on the leaderboard. All three LR variants reach about 0.896 CV accuracy and 0.902–0.904 on Kaggle. QDA's class-specific covariances add flexibility but cost 0.7 CV points and 1.1–1.8 Kaggle points; the tree is about 3 CV points and 3.6 Kaggle points behind LR. XGBoost (0.901) closes most of the tree's gap but stays below LR on CV (−0.8 points) and validation (−1.3). The blends score best on Kaggle (0.907, 0.905) but slightly below LR on CV (0.890) and validation (0.880), so their leaderboard lead of 0.001–0.003 is not confirmed by our own measures. This matches the near-linear boundaries seen in the EDA: extra flexibility mostly fitted noise where the classes overlap.

Table 11: Confusion matrix of the LR baseline on the clean validation set (1,500 rows). Rows: true class; columns: predicted class.

True \ Predicted	Sedan	SUV	Sports Car
Sedan	**423**	46	39
SUV	53	**444**	1
Sports Car	33	2	**459**

7.2 Sedan Is the Bottleneck for Every Model

For the LR baseline, 171 of the 174 validation errors involve Sedan (176 of 180 for the 52 : 48 blend), while SUV and Sports Car are confused only three times (Table 11). Sedan F1 is 0.83 for LR and the blends, 0.82 for QDA and 0.81 for XGBoost, against 0.89–0.92 for the other classes. For the tree at noise 0.2, Sedan F1 is 0.785 and 226 of its 233 errors involve Sedan. Sedans lie between SUVs and sports cars on both power and size, so these errors come from the data rather than from any one model.

7.3 Robustness Differs by Model Family

With 0.2 SD of noise added to the validation features, LR lost 0.7–1.1 points (baseline: 0.8840 → 0.8733, mean of 10 noise draws), QDA 1.3–1.8 points and the tree 0.8 points (0.8487 → 0.8412). XGBoost lost 0.8 points and the 52 : 48 blend 1.1 (0.8800 → 0.8695), about the same as LR, so

blending added no robustness. QDA's full covariance estimates react most to perturbed inputs. The tree's averaged inputs keep it stable, but it starts about 3.5 points lower; on the same rows and noise draws the blend stays about 3 points above it (clean 0.880 vs 0.849, noise 0.3 0.859 vs 0.832). LR therefore keeps the highest accuracy under noise.

7.4 Noise Training Gives Small Gains, Largest for QDA

- **QDA:** the noise-trained model beat the clean-trained one at every noise level in CV and raised the public score by 0.007, from 0.886 to 0.893, the largest gain from any single change in the project.
- **Tree:** noise training raised the robust score, averaged over three CV splits, from 85.3% to 85.8%.
- **LR:** the CV gain stayed within one standard deviation (0.8960 vs 0.8955), and on Kaggle the noise-trained model scored 0.903 against 0.904. LR's linear boundary is already smooth, so it has little to gain.

8 Best Model

Final Model

The best public Kaggle score, **0.907**, came from the **soft-voting blend of logistic regression and XGBoost** with equal weights. The final submission notebook reproduces the **52 : 48** variant of the same blend (public score 0.905).

- **Components:** the LR baseline pipeline (`StandardScaler` + multinomial LR, L2, $C = 1.0$, `lbfgs`, `max_iter` = 1600) and XGBoost (290 trees, depth 5, learning rate 0.042, row and feature sampling 0.8, L1 0.4, L2 8).
- **Combination:** $P = w\,P_{\mathrm{LR}} + (1 - w)\,P_{\mathrm{XGB}}$ with $w = 0.5$ (best public score) or $w = 0.52$ (final notebook); the predicted label is the class with the highest blended probability.
- **Inputs:** all 20 numeric features; `vehicle_id` excluded.
- **Training data:** both models retrained on all 7,500 rows to predict the 2,500 unmodified test rows.
- **Scores (52 : 48 variant):** CV accuracy 0.8903 ± 0.0086 (macro F1 0.8908); validation accuracy 0.8800 (macro F1 0.8804); validation accuracy at noise 0.2 of 0.8695 ± 0.0042 over 10 draws; public Kaggle score 0.905 (50 : 50 variant: 0.907).
- **Reproducibility:** scikit-learn 1.9.1 and XGBoost 3.4.1 are pinned, and a check confirms that the notebook regenerates the submitted file row for row.

Why this model. The blend has the best public score and combines two different kinds of model: LR's smooth linear boundary and XGBoost's threshold rules. The evidence in its favour is, however, small. On CV and validation the LR baseline alone is slightly better (0.8955 and 0.8840), and the blend's leaderboard lead over LR (0.001–0.003) amounts to a few test rows. We therefore regard the blend and the LR baseline as statistically tied: the blend is our final submission because it scored best on the leaderboard, and LR remains the strongest and simplest single model.

9 Key Insights and Limitations

Key insights

1. Removing the 2,500 hidden duplicates before splitting was essential; otherwise copies of one vehicle would appear on both sides of a split and inflate every score.
2. A simple, scaled linear model was enough: on CV and validation no other model or blend beat it, and blending LR with XGBoost raised the public score only from 0.904 to 0.907.
3. Sedan is the hard class for every model because it lies between SUVs and sports cars on both power and size.
4. Noise training helps most where the model is noise-sensitive (QDA, trees) and adds little to LR.
5. For trees, averaging correlated features is an effective way to cancel measurement noise; boosting many shallow trees (XGBoost) closes most of the remaining gap to LR.
6. Reproducing a submission requires pinned library versions: retraining the same LR + XGBoost recipe with a newer XGBoost changed 15 of the 2,500 test predictions.

Limitations.

- The size and form of the test noise are unknown; our levels (0.1–0.3 SD) and independent Gaussian noise per feature are assumptions.
- Some robustness comparisons for noise-trained LR and QDA use a single noise draw, and the noise-trained QDA CV used all 7,500 rows instead of the 6,000 training rows.
- Leaderboard differences of 0.001–0.003 are within test-sample noise, and the size of the public leaderboard subset is unknown. In particular, the blend weights (50 : 50 vs 52 : 48) and the blend's lead over LR were judged mainly on the leaderboard; CV and validation cannot separate them.
- XGBoost results depend on the library version, so CV and validation numbers for the XGBoost-based models were computed with one fixed set of versions (Table 10, note c).
- The overlap between Sedan and the other classes limits the accuracy of every model we tried.

10 Conclusion

We built a complete pipeline for robust three-class vehicle classification, from duplicate removal and EDA to nine models and their Kaggle submissions. A scaled logistic regression was the most accurate single model on CV and validation and among the most robust, reaching 0.904 on the public leaderboard. Blending it with XGBoost gave the best public score, 0.907 (0.905 for the 52 : 48 variant reproduced in the final notebook), although CV and validation show the blend and LR to be statistically tied. Three practices mattered more than model complexity: removing hidden duplicates before any split, testing every model on noisy as well as clean data, and averaging over repeated noise draws and CV folds. Almost all remaining errors involve Sedan, so further gains are more likely to come from features that separate sedans than from more flexible classifiers.